State-Aware Fuzzing of JavaScript Engines with LLM-Guided Instrumentation

Wai Kin Wong*
Hong Kong University of Science and Technology
Hong Kong, China
wkwongal@cse.ust.hk

Dongwei Xiao*
Hong Kong University of Science and Technology
Hong Kong, China
dxiaoad@cse.ust.hk

Cheuk Tung Lai
VX Research Limited
London, United Kingdom
darkfloyd@vxrl.hk

Ping Fan Ke
Singapore Management University
Singapore, Singapore
pfke@smu.edu.sg

Shuai Wang†
Hong Kong University of Science and Technology
Hong Kong, China
shuaiw@cse.ust.hk

arXiv:2609.24550v1 [cs.CR] 21 Sep 2026

Abstract

The security of the modern web depends on the correctness of JavaScript (JS) engines, yet these complex systems remain vulnerable to high-impact bugs. A critical limitation of state-of-the-art fuzzers is the coverage plateau: once a fuzzer saturates the control-flow graph, edge coverage loses its ability to guide discovery. Because complex engine behaviors, such as JIT optimization tiers and hidden class transitions, often share identical edge coverage, standard coverage metrics are blind to the distinct internal states required to trigger deep errors. To bridge this gap, we present StateLens, a framework that employs Large Language Models (LLM) to automate the discovery of deep internal states. Blindly placing instrumentation probes at all states is infeasible due to the vast state space and the high runtime overhead. StateLens introduces a novel agent-based reasoning pipeline that emulates the intuition of a security researcher. By iteratively traversing code and developer comments, our agents intelligently select high-value instrumentation targets, effectively separating logic-driving states from irrelevant data. This results in synthesizable, high-signal feedback probes that map the engine's hidden configurations. This instrumentation feeds a dual-feedback mechanism, effectively guiding the fuzzer toward unexplored engine semantics. Our evaluation confirms that StateLens significantly outperforms state-of-the-art fuzzers and uncovers 68 new bugs.

CCS Concepts: • **Security and privacy → Browser security**.

Keywords: JavaScript engine fuzzing; fuzz testing; state coverage; large language models; program analysis; program instrumentation; vulnerability discovery

ACM Reference Format:

Wai Kin Wong, Dongwei Xiao, Cheuk Tung Lai, Ping Fan Ke, and Shuai Wang. 2026. State-Aware Fuzzing of JavaScript Engines with LLM-Guided Instrumentation. In *ACM SIGOPS 32nd Symposium on Operating Systems Principles (SOSP '26), September 29-October 02, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3830418.3843900

1 Introduction

The security of today's software stack increasingly hinges on the security of JavaScript (JS) engines. Once confined to browsers, JS engines now underpin platforms ranging from web clients and server-side runtimes such as Node.js [16], Deno [9], and Bun [8] to serverless and edge platforms like AWS Lambda [7] and Cloudflare Workers [18]. Their ubiquity makes them a prime attack surface: a single vulnerability can put billions of users or entire application stacks at risk through malicious web content alone. Indeed, nearly one-third of in-the-wild exploits in 2024 targeted JS engines [37].

Discovering deep vulnerabilities in JS engines remains challenging. *Structural coverage* like line and branch coverage has long been the primary feedback signal for JS engine

*Equal contribution.
†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/2026/09
https://doi.org/10.1145/3830418.3843900

fuzzers [40, 42]. However, reaching a code path is often insufficient to trigger a bug. Many high-severity JS-engine bugs stem from logic errors in internal state management and manifest only under specific runtime conditions. Even with identical code paths, semantically distinct internal states can lead to different outcomes.

Unlike structural coverage, which captures *which code* executes, *state coverage* [23, 56] distinguishes *under what runtime conditions* that code executes. Prior approaches, however, are difficult to scale to JavaScript engines. They either require manual state selection, or are agnostic to the semantics of JS engines. These approaches fail to capture the complex, cross-component interactions that often harbor deep engine bugs.

The key challenge is deciding *which* states to instrument in a vast internal state space. Exhaustive instrumentation is infeasible, yet sparse instrumentation leaves the fuzzer blind. We observe that developers have already encoded this knowledge in comments, design documents, historical bug reports, and debug assertions that describe which states are fragile and why. Given LLMs' ability for semantic understanding, they are well-suited to extract this knowledge and identify semantically meaningful states for instrumentation. We thus introduce *StateLens*, a framework that for the first time, enables state coverage for JS engines with the power of LLMs for automated state identification and instrumentation.

StateLens operates in three stages. In the *analysis stage*, an LLM agent guided by a queryable knowledge base of developer artifacts performs iterative call-graph and dataflow traversal to identify semantically meaningful state expressions, transitions, and cross-component interactions. In the *instrumentation stage*, it synthesizes lightweight, read-only probes that project these states into a compact shared-memory bitmap. During the *fuzzing stage*, a dual-feedback mechanism integrates state coverage into the fuzzing loop alongside traditional edge coverage, retaining inputs that exercise previously unseen state combinations even when edge coverage reports no change. In summary, this paper makes the following contributions:

- Conceptually, this work is the first to automatically identify and instrument JavaScript engine states as fuzzing feedback. For complex systems like JavaScript engines, reaching code/branch alone is insufficient to explore the rich semantic space in the engine. Furthermore, rather than relying on human experts to identify critical states for instrumentation, we propose to automatically mine such states with LLM agents to enable scalable state-aware fuzzing.
- Technically, we design and implement StateLens, a three-stage instrumentation pipeline that automatically identifies behavior-relevant state expressions from developer-authored artifacts and synthesizes validated probes in production JS engines, together with a dual-feedback adapter that integrates semantic state projections into the fuzzing loop alongside traditional edge coverage.
- Experimentally, we evaluate StateLens on six major JS engines (V8, SpiderMonkey, JavaScriptCore, QuickJS, Hermes, and Escargot) and discover 68 new bugs in a long-running fuzzing campaign. In a 72-hour comparison with baseline fuzzers, StateLens found 70% more bugs than the best baseline fuzzer.

2 Background

JS Engines as Critical System Infrastructure. JS engines have evolved from browser components into foundational infrastructure for modern computing systems. In desktop software, frameworks such as Electron embed engines like V8 to power widely deployed cross-platform applications, including Visual Studio Code, Slack, and Discord. In server environments, runtimes such as Node.js and Deno underpin backend services, cloud functions, and microservices. Major cloud providers further rely on managed JS runtimes to execute cloud and edge workloads, including AWS Lambda, Google Cloud Functions, and Cloudflare Workers. Vulnerabilities in JS engines have been repeatedly weaponized to achieve remote code execution (RCE) across fleets of Electron-based desktop applications [22, 97], while sandbox-escape bugs in server-side JS runtimes have threatened cross-tenant isolation in serverless cloud architectures [21]. The security and reliability of modern desktop and cloud systems therefore hinge on the robustness of JS engines, making effective fuzzing of these complex systems a critical priority.

JS Engines Architecture. The internal architecture of modern JS engines is inherently complex. These engines are highly stateful, multi-stage software systems comprising interpreters, multi-tier Just-In-Time (JIT) compilers, concurrent garbage collectors (GC), and WebAssembly runtimes. Moreover, their execution behavior can change drastically with the internal state. For instance, JS engines' memory management subsystems often employ sophisticated heap allocation and garbage collection strategies that are sensitive to current memory pressure and object models (e.g., hidden classes or shapes). This intricate interplay between dynamic compilation, memory management, and execution creates a vast space where vulnerabilities may arise only under highly specific, history-dependent sequences of runtime events.

Fuzzing JS Engines. Fuzzing has become a primary technique for vulnerability discovery in complex software [36,

49, 64], and has been widely adopted for testing production JS engines [83, 86]. However, directly applying general-purpose fuzzers like AFL [99] is largely ineffective; such tools mutate raw bytes and rely on basic structural control-flow edges as mutation guidance, which is insufficient to navigate JS's massive syntactic input space or trigger deeply nested compilation states. Consequently, the state of the art has shifted toward domain-specific fuzzers, such as Fuzzilli [40], DIE [70], and Superion [85], that operate on custom intermediate representations (e.g., FuzzIL) or carefully preserve semantic properties during mutation to generate valid programs. Yet, even with these specialized generators, uncovering deep bugs remains computationally expensive and many vulnerable states are still systematically missed.

3 Motivation

Existing JS engine fuzzers commonly rely on structural coverage metrics, such as edge coverage, to guide input generation. In this section, we first use a real-world vulnerability to illustrate why structural coverage is insufficient. We then explain why existing approaches to state coverage do not scale to JS engines, before presenting observations that inform StateLens's state coverage design.

3.1 The Limits of Structural Coverage

Structural Coverage Plateau. Modern JS engines, such as V8 and SpiderMonkey, comprise millions of lines of C++ code. The same control-flow paths are repeatedly exercised under different object layouts, optimization tiers, heap conditions, and cache configurations. Existing JS engine fuzzers [40, 70, 84, 96] guide input generation with structural coverage metrics, such as edge coverage, retaining inputs that explore new control-flow edges. However, once these edges saturate, executions that reach new internal conditions may provide no new feedback.

```
1 function main(i) {
2   class C { m() { return super.x; } }
3   let rect = new DOMRect(1, 1, 1, 1);
4   C.prototype.__proto__ = (i < 19) ? {} : rect;
5   rect.x;      // cache a handler for DOMRect.x
6   new C().m(); // reuse it, but invoke on C
7 }
8 for (let i = 0; i < 20; i++) main(i);
```

Case Study: CVE-2022-1134. We illustrate this limitation with CVE-2022-1134 [4], a high severity vulnerability in V8. V8 uses Inline Caches (ICs) to accelerate object property accesses. Line 5 accesses `rect.x`, causing V8 to cache a handler for the native `DOMRect.x` accessor. In the first 19 iterations, `C.prototype.__proto__`, which determines the superclass prototype of `C`, points to an empty object (line 4), so the cached handler is never invoked when `super.x` is called on a `C` instance (line 6). In the final iteration, `super.x` starts its lookup from the `DOMRect` object and therefore reuses the cached native accessor handler. However, JavaScript `super` semantics preserve the current `C` instance as the accessor receiver. V8 applies a handler specialized for a `DOMRect` receiver to an incompatible `C` instance, causing type confusion.

The bug stems from the mismatch between IC handler construction and execution phases. Handler construction checks the `Map`, which is V8's runtime object-layout descriptor, of the lookup-start object, whereas handler execution invokes the native accessor on `p->receiver()` without checking that receiver's Map. The cached handler is therefore accepted based on the `DOMRect` prototype but executed on a `C` instance.

```
1 // Handler construction: LoadIC::ComputeHandler
2 Handle<Map> map = lookup_start_object_map();
3 ...
4 LookupHolderOfExpectedType(map, &holder_lookup);
```

```
1 // Handler execution: AccessorAssembler::
      HandleLoadAccessor
2 ...
3 CallApiCallback(..., p->receiver());
```

The Missing Feedback Signal. A fuzzer guided by structural coverage readily covers all the lines above. The vulnerability instead depends on a relationship that edge coverage cannot represent. In the vulnerable execution, the cached handler is compatible with `lookup_start_object` to pass the validation in handler construction, but incompatible with `receiver` to cause a type confusion in handler execution. Consequently, safe and vulnerable executions can traverse the same control-flow edges while differing in whether the cached handler is compatible with the actual receiver.

3.2 State Coverage as Fuzzing Feedback

To overcome the limitations of structural coverage, state coverage can effectively distinguish executions that traverse the same control-flow edges but differ in internal state.

Inapplicability of Prior State-Coverage Approaches. Although prior works have explored state coverage, these approaches are not directly applicable to JS engines. IJON requires developers to manually identify and annotate the states to track [23], while SDFuzz represents states using call-stack configurations [56]. Manual annotation does not scale to JavaScript engines, whose millions of lines of code and tightly interacting subsystems expose an enormous number of potential states. Call-stack configurations, meanwhile, capture calling context but not semantic runtime conditions, and therefore cannot distinguish executions that share the same call stack but differ in critical variable values or object configurations.

The core challenge is therefore to automatically and effectively identify JS engine states to instrument. To understand what states facilitate bug discovery, we analyzed 35 distinct vulnerabilities from Google's public v8CTF tracker [38] that

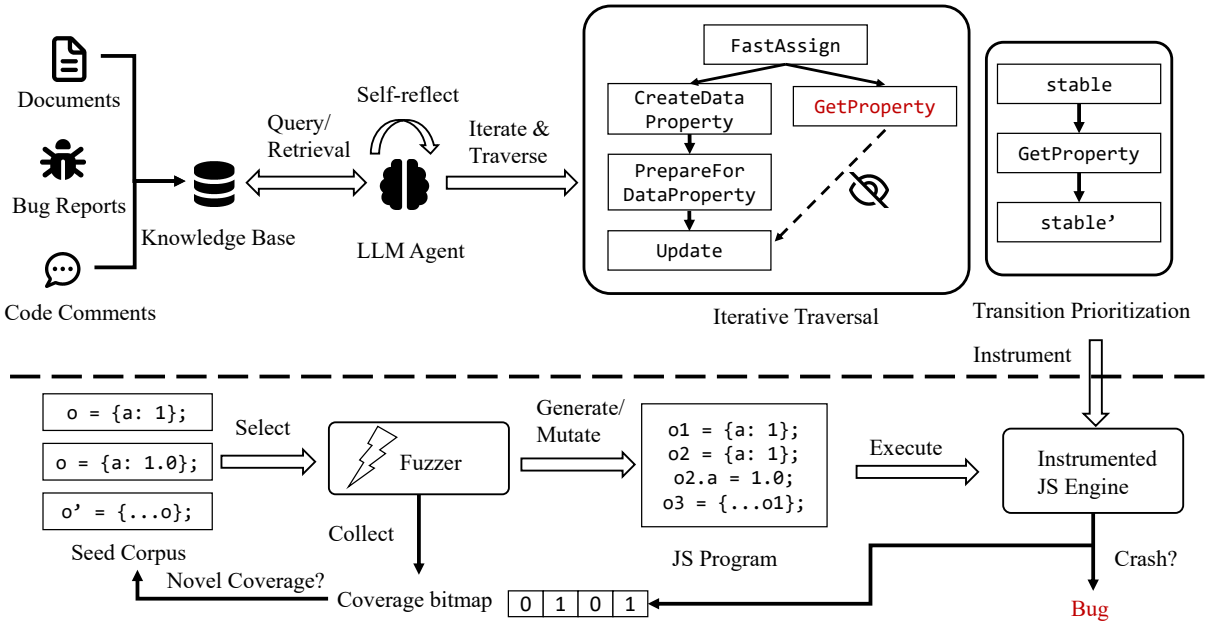


Figure 1. The overall architecture of StateLens, which consists of two main phases: (1) an offline analysis and instrumentation phase guided by an LLM agent that extracts semantic beacons from developer artifacts and source code to identify critical states; and (2) an online fuzzing phase where the instrumented engine provides dual feedback to guide input generation toward unexplored semantic states.

were fixed by December 31, 2025. Each entry includes a working exploit [39]. For each case, two authors manually inspected the exploits and corresponding patches and resolved classification disagreements through discussion. We derive three observations from this study that inform StateLens's state coverage design. These observations are not mutually exclusive.

O1: Bug-Relevant Conditions Cross Implementation Boundaries. In 31 of 35 cases (88.6%), a condition established in one function or engine phase was invalidated or consumed in another. In issue 391907159 [20], V8's Wasm code garbage collector marked an import wrapper as dying, but the wrapper cache reused it before reclamation completed. The wrapper was then freed while still referenced, causing a use-after-free.

O2: Side Effects and Internal Transitions Invalidate Assumptions. In 12 of 35 cases (34.3%), a side effect or engine-internal transition changed state on which other code still relied. These transitions involved object layout, compiler tier, allocation, garbage collection, and value representation. In issue 400052777 [19], an operation reached an object through a different reference and changed its map through an elements-kind transition. TurboFan retained its earlier layout inference, causing downstream optimization to produce a type confusion.

O3: Developer Artifacts Reveal Semantically Important States. Engine developers record critical state indicators in assertions, typed enumerations, comments, design documents, and bug reports. We call these artifacts *semantic beacons*, which are not itself states, but rather developer-authored evidence that helps StateLens identify states to instrument. For example, V8 contains more than 33,000 debug assertions and 772 enum class definitions, including engine concepts such as optimization tiers, IC states, and allocation modes. These signals provide scalable starting points for identifying behavior-relevant state dimensions without instrumenting every variable.

3.3 Combining Semantic Reasoning with Fuzzing

Automatically extracting these semantic beacons at scale is a complex task. Assertion macros span multiple preprocessor layers, and critical context is frequently embedded in natural language comments rather than machine-readable abstractions. Traditional static analysis tools struggle with this interplay of cross-file dependencies and informal developer intent. Recent advances in applying Large Language Models (LLMs) to program analysis suggest that these artifacts can be mined automatically. Prior work shows that LLMs can infer useful program invariants from code [71] and synthesize semantically meaningful predicates and instrumentation for fuzzing [108].

Enabling LLMs for state-aware fuzzing does not come without challenges. Directly applying LLMs to the entire JS engine codebase is infeasible due to the sheer size of the code and the complexity of the interactions between components. Instead, we use an LLM agent for semantic selection, while grounding its reasoning with retrieval and conventional call-graph and data-flow tools. The LLM is used offline for semantic reasoning, while the fuzzer remains responsible for high-throughput exploration and concrete bug triggering.

4 Overview

Fig. 1 shows two stages: offline LLM-guided instrumentation and online state-aware fuzzing. Offline, the agent analyzes a knowledge base to synthesize targeted probes for critical internal states. Online, dual feedback rewards inputs that exercise new semantic states after edge coverage saturates, guiding the fuzzer toward bugs that conventional coverage feedback misses.

We define a *state* as a side-effect-free expression over runtime values, which can be variables or side-effect-free functions (observation functions). For an operation with pre- and post-operation program points, the ordered pair of states at those points defines a *state transition*. *State coverage* is the set of previously unseen states or transitions recorded in the semantic bitmap. Two executions contribute different coverage when they produce different observed values at the same site, subject to bitmap hashing.

4.1 Semantic Beacon Extraction

A *semantic beacon* is itself not a state, but evidence from developer artifacts suggesting a potential critical state or transition to instrument. For example, an assertion may encode an invariant over an object's storage mode, while a bug report may describe a transition that violated such an invariant. Semantic extraction aims to mine these beacons from developer artifacts and map them to relevant states. StateLens performs this extraction in three steps:

Knowledge Base Construction. Developer artifacts such as design documents, in-source comments, and historical bug reports record which internal states matter and which subsystems manage them. StateLens aggregates these sources into a vector-indexed *knowledge base* (KB); Section 5 gives further construction details.

Knowledge Retrieval. The agent consults the KB on demand rather than loading all artifacts at once. At each discovery step, it issues a targeted natural-language query and retrieves ranked snippets that connect a semantic feature to relevant files and functions.

Self-Reflection Filtering. A query can retrieve snippets that use the right words for the wrong entity or transition. Through self-reflection prompting [63], we instruct the agent to inspect the retrieved snippets, justify why they qualify for the semantic beacon, and output `keep` or `reject`.

Running Example. We use CVE-2024-5830 [5] as a running example to show how StateLens performs semantic beacon extraction.

Listing 1. V8's map update and fast-mode assertion.

```
1 Handle<Map> Map::PrepareForDataProperty(...) {
2   map = Update(isolate, map); // replace outdated layouts
3   DCHECK(!map->is_dictionary_map()); // expects fast mode
4   ...
5 }
```

V8 associates each JS object with an internal layout descriptor, called a `Map`. Besides storing object properties, the descriptor records whether object properties use a compact array representation ("fast mode") or a hash-table representation ("dictionary mode"). The code in Listing 1 prepares such a descriptor before adding a property. Its assertion expresses the assumption that after `Map::Update` updates an outdated descriptor, the `Map` should still use fast-mode storage.

A historical bug report [3] documents that `Update` can return a dictionary map when the hidden-class transition table is exhausted. While such a bug is already fixed, it inspires the agent to generalize the underlying bug pattern by paying attention to transitions across `Map::Update`, as changing a map from fast to dictionary mode may be security-relevant.

Listing 2. Phase 1 KB query and representative results.

```
① <KB query> "Map::Update map storage mode transition"
<Retrieved snippets>
  [1] comment     an outdated map may cache the map to which
                  objects should migrate.
  [2] comment     walks parent links to test whether maps
                  belong to the same context.
  [3] design doc  describes the proposed FixedMap collection
```

The agent then queries the KB for evidence connecting `Map::Update` to changes in map storage mode. The query returns three candidates. Candidate [1] explains that an outdated map may cache the map to which objects should migrate, helping establish how `Map::Update` can replace the caller's original map. Candidate [2] also concerns V8 maps, but only checks whether maps in a transition tree belong to the same native context. Candidate [3] describes the ECMAScript `FixedMap` collection, where "map" refers to a JavaScript collection rather than V8's internal object-layout descriptor.

Only Candidate [1] is relevant to the target transition. Candidate [2] concerns transition-tree membership rather than storage mode, while Candidate [3] refers to a different abstraction altogether. Because keyword overlap alone cannot distinguish these cases, StateLens prompts the LLM to interpret each snippet against the target fast-to-dictionary

transition and justify whether it should be retained, with the results are shown in Listing 3.

Listing 3. Phase 1 self-reflection: resolving the retrieved snippets against the target storage-mode transition.

```
② <Self-reflection>
Target: map storage mode across Update()

  keep   [1] links the old hidden-class map to its
             migration target
  reject [2] checks native-context membership in the
             transition tree, not storage mode
  reject [3] describes an ECMAScript FixedMap,
             not V8's internal hidden class
```

The LLM then combines three pieces of evidence. The source assertion shows that `Map::PrepareForDataProperty` expects the result of `Map::Update` to remain in fast mode; the historical bug report shows that this expectation can be violated; and Candidate [1] explains how an outdated map can be replaced through the migration path. Together, these sources yield the beacon summarized in step ③ of Listing 4.

Listing 4. Phase 1 Beacon Summary for the running example.

```
③ <Beacon Summary>
  state descriptions: map deprecation, storage mode
  transition hint  : deprecated fast map
                     -> dictionary map via Map::Update
  seed symbols     : Map::PrepareForDataProperty,
                     Map::Update
```

At this point, StateLens has identified the relevant states using natural language descriptions, the suspected transition, and the seed functions from which to continue analysis. Phase 2 starts from these symbols and traces the interprocedural path by which a deprecated fast map can be updated into a dictionary map.

4.2 Phase 2: Iterative State Discovery

Phase 2 builds on semantic beacon extraction by expanding seed source symbols to track where a target state is established, modified, and consumed. This interprocedural tracing is based on the observation O1 in Section 3.2 that bug-relevant conditions often span multiple functions and are rarely visible from a single source location.

To accomplish this, the agent iteratively explores a frontier of functions and variables, guided by the Beacon Summary's state descriptions and transition hints. It utilizes call-graph traversal, data-flow analysis, source inspection, and KB retrieval to uncover hidden semantic contexts and track states across functions.

Running Example. The agent expands the fully qualified seed symbols in the Beacon Summary. Because C++ function names can be reused, it checks each call-graph candidate against the source and retains only edges that invoke the target function. For the running example, the validated edges form the two branches shown in step ① of Listing 5.

Listing 5. Evidence-guided call-graph expansion and query refinement.

```
① CallGraph(Map::Update)

source-validated direct callers
  Map::PrepareForDataProperty
  JSObject::MigrateInstance

source-validated caller chain from
  Map::PrepareForDataProperty
  <- TryFastAddDataProperty
  <- CreateDataProperty
  <- FastAssign

The migration branch supplies the next query

  ② <KB query> "JSObject MigrateInstance
     deprecated map migration"
```

Query Refinement. The migration branch contributes the `JSObject::MigrateInstance`. The agent combines this symbol with the beacon's deprecated map context in the refined query shown in step ② of Listing 5. Later searches can then focus on the migration code that changes map storage.

Listing 6. State Report for the running example.

```
③ <State Report>
WHAT : whether Map::Update converts a deprecated
       fast map into a dictionary map.
WHY  : TryFastAddDataProperty later uses the returned
       map in descriptor and write logic that assumes
       fast-property storage.
HOW  : an accessor can deprecate the target transition
       map before CreateDataProperty reuses it; updating
       that map may produce dictionary-mode storage.
Candidate sites
       immediately before and after Map::Update in
       Map::PrepareForDataProperty
```

From Traversal to a State Report. The two branches meet at `Map::Update`. The migration branch follows `JSObject::MigrateInstance` into the deprecated map update path. The caller branch ascends from `Map::PrepareForDataProperty` via `TryFastAddDataProperty` and `CreateDataProperty` to `FastAssign`; it exposes the downstream fast-map use and the accessor that establishes the transition precondition. For this example, the branches ground the state-changing operation, its downstream use, and their concrete source locations. A partial report produced at the step limit still undergoes self-reflection; reports with no surviving locations are discarded. Step ③ in Listing 6 shows the resulting report.

Listing 7. Simplified excerpt of V8's `FastAssign` (`js-objects.cc`). The `stable` flag tracks source-map changes during property reads; the bug-relevant update occurs later on the target.

```
bool stable = true;
for (InternalIndex i : map->IterateOwnDescriptors()) {
  if (stable) {
    // Fast path: decode directly from descriptor array
    if (details.kind() == kData) {
      prop_value = FastPropertyAt(from, ...);
    } else {
      // Getter: may trigger side effects
      prop_value = Object::GetProperty(&it);
      stable = from->map() == *map;  // map changed?
    }
  } else {
    // Slow path: fresh lookup
    prop_value = Object::GetProperty(&it);
  }
  CreateDataProperty(target, next_key, prop_value);
}
```

Causal Chain. The caller branch reaches `FastAssign` (Listing 7). Before cloning, a transition map already exists for adding the copied property. `FastAssign` evaluates an accessor before calling `CreateDataProperty`. By creating a conflicting field representation, the accessor deprecates that transition map. `TryFastAddDataProperty` then reuses the deprecated map and passes it to `Map::PrepareForDataProperty`. `Map::Update` may return a dictionary map, yet the following `WriteToField` still assumes fast-property storage, causing the type confusion [5].

4.3 Phase 3: Transition Prioritization and Probe Synthesis

Side-Effect-Driven Transitions. O2 in Section 3.2 motivates StateLens to prioritize state changes caused by side effects. In `FastAssign`, an accessor can deprecate the target transition map before `CreateDataProperty` reuses it. Edge coverage follows the same path whether `Map::Update` returns a fast or dictionary map.

Selecting Relevant Transitions. StateLens follows call-graph and data-flow links from each report, then selects in-scope expressions that observe the reported state without side effects.

Running Example. The analysis exposes three candidates: `stable`, `is_deprecated()`, and `is_dictionary_map()`. The first directly represents source-map stability, but it does not directly reflect the selected target-map property; StateLens therefore rejects it. The other two expressions capture the target map's transition precondition and downstream outcome. Recording storage mode on both sides of `Map::Update` further distinguishes the bug-relevant fast-to-dictionary update from a map already in dictionary mode. This combination of states captures the update and the later fast-storage assumption. Step ① in Listing 8 shows the selected expressions as states to instrument; step ② shows the synthesized probe.

Listing 8. Selected states and the synthesized probe.

```
① Selected transition
  [src/objects/map.cc, <before Update>,
   <map->is_deprecated(),
    map->is_dictionary_map()>]
  [src/objects/map.cc, <after Update>,
   map->is_dictionary_map()]

② Synthesized probe
  bool deprecated_before = map->is_deprecated();
  bool dictionary_before = map->is_dictionary_map();
  map = Update(isolate, map);
  uint32_t pre_state =
      (deprecated_before << 1) | dictionary_before;
  SEMANTIC_CONTEXT_ENUM(id, pre_state,
                        map->is_dictionary_map());
```

The probe packs the two pre-state predicates into one value and pairs it with the post-update storage mode. The resulting bitmap entry distinguishes the deprecated-fast-to-dictionary transition from other outcomes of `Map::Update`.

4.4 Instrumentation Stage

Phase 3 outputs the source locations and state expressions to instrument. The instrumentation stage translates these specifications into code patches that expose the selected transitions through lightweight probes.

Instrumentation Design. As mentioned in Section 4.3, critical feedback signals often arise from state transitions. To capture these transitions, StateLens inserts temporary buffers into the engine that record pre-transition values of the relevant state expressions, then hashes the combination of the pre- and post-transition values into a shared-memory bitmap that the fuzzer can read. This design allows the fuzzer to recognize when a critical state transition occurs, even if the control flow remains unchanged.

Instrumentation and Memory Management. Given the selected state expressions and their source locations, probe insertion first adds temporary buffers for pre-transition values and then emits probe calls based on each observed value's type. At JS engine startup, a dedicated initialization routine creates or opens a POSIX shared memory region keyed to the process ID, with a fixed layout comprising the bitmap and the maximization slot array (for tracking integer states). This region is mapped into the engine's address space and into the fuzzer's address space, enabling zero-copy state transfer. The region is zeroed at initialization and remains writable throughout the execution lifetime.

4.5 Dual-Feedback Fuzzing

During fuzzing, generated JS programs are executed on the instrumented engine, and those that trigger novel bitmap entries are retained for further mutation. We use a dual-feedback design with two instrumented JS engine instances that share the same seed corpus: one instrumented for structural coverage only, and one instrumented for both structural and state coverage. The fuzzer dynamically switches between them based on structural coverage growth, leveraging the strengths of both signals at different stages of the search process while avoiding state-instrumentation overhead when it is not yet useful.

The structural-only instance is cheaper to execute and is therefore used for early exploration. This lets the fuzzer quickly explore new code paths with dense and informative signal while the input space remains largely unexplored. The fuzzer continuously monitors structural coverage growth and, once it plateaus for long enough (details in Section 5), switches to the state-augmented instance. This enables it to use finer-grained state signal to guide the search toward inputs that trigger specific state transitions, which are often necessary to expose deep bugs invisible to structural coverage.

5 Implementation of StateLens

Instrumentation and Feedback. We compile the instrumented engine to identify syntax, type, and scope errors, run its existing test suite to detect unintended side effects such as incorrect reference-count updates, and prompt the LLM to repair whatever either step reports.

We run the fuzzer and the target engine as separate processes, so the fuzzer cannot perturb the engine's internal state and a crashing or hanging engine cannot disrupt the fuzzing service. To reduce the overhead of inter-process communication (IPC), we implement a shared-memory region that the target engine can write to when an instrumentation probe is triggered, and the fuzzer can read from it after each test execution for feedback collection. We implement a small set of probe utility functions that the instrumentation can call to write to the shared-memory bitmap. For example, `sem_set` takes an index and sets the corresponding bit in the bitmap. By design, these functions are read-only with respect to engine state and write exclusively to the shared-memory bitmap, and thus shall not cause side effects on the engine's normal execution.

Integration with Existing Fuzzers. We implement StateLens on top of Fuzzilli [40], a coverage-guided fuzzer whose modular, multi-engine design supports our evaluation across different targets. Within Fuzzilli, we switch to state-augmented feedback after 100 consecutive iterations yield no new structural edge; discovering an edge resets the counter. We use this threshold for every engine and run. In total, we wrote 397 lines of C to support the instrumentation and feedback infrastructure, and 2774 lines of Swift to implement the IPC channel between the fuzzing module and the target engine binary, enabling collection of runtime feedback and propagation of state information back to the fuzzer.

Table 1. Code analysis tools available to the agent.

Tool	Input	Output
CallGraph	Function name	Callers and callees
DataFlow	Variable, scope	Def-use sites
ASTMatch	Structural pattern	Matching AST nodes

Knowledge Base Construction. To construct the knowledge base in Section 4.1, we crawl three source types for each engine besides source code: (1) security-labeled reports from the engine's bug tracker, (2) in-source comments extracted via AST traversal of the engine's source code repository, and (3) developer design documents such as ECMAScript specifications and engine-specific design docs. Documents are chunked and indexed into a vector store for retrieval-augmented queries.

Program Analysis Tools. In order to enable efficient navigation of the codebase, we implement a set of program analysis tools that the agent can use to query the codebase. Besides simple file search and text grep, we implement three tools for interprocedural call-graph traversal, def-use tracking, and structural pattern matching. These tools are listed in Table 1: (1) CallGraph tool enables the agent to perform interprocedural call-chain traversal, which facilitates tracing the flow of states across function boundaries. (2) DataFlow tool allows the agent to track the definitions and uses of variables, enabling it to identify how states are defined and propagated through the code. (3) ASTMatch tool enables the agent to perform structural pattern matching across the codebase, which can help identify code regions that manipulate states via certain patterns. To bound the exploration cost, we cap each semantic beacon at 25 tool-using steps. Nonetheless, these tools are not the only means for the agent to navigate the codebase; it can also leverage the KB to find relevant code snippets. We also admit that while more advanced program analysis tools such as symbolic execution could potentially be helpful for the agent to understand the codebase, they are not currently implemented in our prototype due to engineering complexity and potential scalability issues. We leave the integration of more advanced program analysis tools as future work.

6 Evaluation

We evaluate StateLens on six JavaScript engines to answer three questions:

- **Q1:** Can StateLens find real-world bugs in JavaScript engines? (Section 6.2)
- **Q2:** How does StateLens compare to state-of-the-art JS engine fuzzers? (Section 6.3)
- **Q3:** How do individual components contribute to effectiveness? (Section 6.4)

6.1 Evaluation Setup

JS Engines Under Test. We evaluate StateLens on six JS engines with different codebase sizes and use case scenarios: V8, SpiderMonkey, JavaScriptCore, QuickJS, Hermes, and Escargot. These engines serve as critical infrastructure beyond just browser runtimes: V8 drives edge compute for major cloud providers [14], SpiderMonkey and QuickJS are embedded in databases such as MongoDB [15] and CouchDB [6], JavaScriptCore underpins server-side runtimes like Bun [8], Hermes is the default engine for React Native [17], and Escargot powers system services on Samsung's Tizen OS for IoT devices.

All the six engines are mature, widely used, and have been continuously stress-tested in the wild, with extensive fuzzing efforts from both internal teams (using tools like OSS-Fuzz and in-house fuzzers) and the security community. Given this, they represent a challenging testbed for evaluating StateLens's bug-finding capabilities, as many of the low-hanging fruits have already been discovered and fixed.

Experimental Environment. All experiments run on an Ubuntu 22.04 LTS workstation with a 64-core Ryzen 9980X processor and 128 GB of RAM. We compile each engine with LLVM 22 [51] and enable AddressSanitizer [74]. Every crashing input is replayed on the same engine revision and sanitizer configuration without StateLens probes, and we count only failures that reproduce. Probe synthesis uses GPT-5.1 [12] at temperature zero to reduce sampling variance.

Instrumentation Cost. One end-to-end instrumentation synthesis takes 19–80 minutes, with an average of 48 minutes across the six engines. QuickJS completes fastest, while V8 takes longest because of its larger codebase. Each synthesis consumes an average of 1.02M input tokens and 0.51M output tokens per engine. The average monetary cost is $6.45, comprising LLM calls ($3.20), retrieval ($1.50), re-instrumentation ($1.00), and validation-driven repair ($0.75). The per-engine cost ranges from $0.28 for QuickJS to $13.75 for V8 and broadly follows codebase size.

Runtime and Memory Overhead. We measure runtime overhead by comparing the end-to-end throughput of Fuzzilli and StateLens on V8 over a 24-hour fuzzing session. Fuzzilli executes 9.35M test cases and StateLens 8.83M, a 5.5% runtime overhead.

For memory, we sample resident set size (RSS) across all 10 QuickJS instances for five minutes after 24 hours of fuzzing. Mean RSS is 23% higher with StateLens probes than without them.

6.2 Bug Finding

Overall Effectiveness. During a three-month continuous fuzzing campaign, StateLens uncovered 68 bugs across six JavaScript engines: 25 in V8, 14 in JavaScriptCore, 9 in Escargot, 7 each in SpiderMonkey and QuickJS, and 6 in Hermes. Throughout the campaign, we periodically updated each engine to its latest release and reinstrumented each new revision, so subsequent runs targeted current code rather than bugs already fixed upstream. These engines have been extensively audited by their vendors, stress-tested by OSS-Fuzz [67], and covered by vendor bug bounty programs [31]; the fact that StateLens still finds new bugs in such mature codebases underscores its effectiveness.

Independent Root Causes. The 68 bugs represent independent root causes: upstream maintainers tracked them in separate tickets and fixed them with distinct patches. These causes span execution tiers and compiler subsystems, including graph construction, IR lowering, register allocation, deoptimization dispatch, and the interaction between JIT constant emission and shared-heap garbage collection.

Security Impact. At least 35 bugs were confirmed to have security implications, including remote code execution and sandbox escapes. The V8 security team and Meta awarded bounties for several discoveries, Apple credited us in security updates, and some bugs were adopted as capture-the-flag (CTF) challenges [13].

Listing 9. Minimized QuickJS PoC.

```
(function(){
  let w = new os.Worker('aaaa');
  w.onmessage = function(){};
  Object.defineProperty(w, 'onmessage', {
    set(v){
      delete w.onmessage;
      w.postMessage = null;
    }
  });
  w.onmessage = 42;  // triggers setter
})();
```

Case Study. We show two of our uncovered bugs in detail to illustrate why state-sensitive probes are crucial for finding them. These bugs are only discovered by StateLens and are missed by all the vendor fuzzers and security community. In fact, these bugs have been present in the codebase for years and have been continuously fuzzed by the respective engine developers, yet they remain undetected until StateLens found them.

QuickJS: Use-After-Free in Worker. Listing 9 shows the PoC for a use-after-free bug in QuickJS's Worker implementation. Line 2 creates a Worker, which registers an internal communication port in the runtime's thread state. Lines 3–9 install an ordinary `onmessage` handler and then redefine `onmessage` as an accessor whose setter deletes that handler and nullifies the Worker's `postMessage` reference. Line 10 assigns to `onmessage`. Because the property is now an accessor, the assignment runs the setter and leaves the Worker's internal port references out of sync with the runtime's port list. When the program exits, the runtime first frees its thread state, which holds the head of the port list, and then runs a final GC pass that invokes the Worker's finalizer. Because the port was never unlinked, the finalizer reaches `js_free_port` (Listing 11), whose `list_del` writes through two neighbor pointers that still reference the freed thread state, producing a use-after-free write.

Two probes expose the state combinations behind this bug. Both use `SEMANTIC_CONTEXT_ENUM`, which takes a site identifier and two engine values and sets the bitmap entry for that triple, so the fuzzer distinguishes *combinations* of engine values at a site rather than individual values. The probe in Listing 10 records the old and new property types at every `defineProperty` redefinition; in the PoC it fires on the operation that converts `onmessage` from a data property into the destructive setter.

Listing 10. Probe 1: property type transition in defineProperty (quickjs.c).

```
1 int JS_DefineProperty(JSContext *ctx, ...) {
2   ...
3   // old type x new type
4   SEMANTIC_CONTEXT_ENUM(0x3144,
5     (old_flags & JS_PROP_TMASK) >> 4,
6     (flags & JS_PROP_TMASK) >> 4);
7   ...
8 }
```

Listing 11 records two state dimensions when the Worker's finalizer runs: the runtime's GC phase and the type of the port's message handler. When a Worker is reclaimed through the normal cleanup path, the handler is cleared before GC collects the object, so the finalizer sees a null handler outside of any GC phase. The PoC preserves a live handler that persists into the GC cycle, stressing a finalizer path that the runtime's cleanup ordering does not anticipate.

A coverage-guided fuzzer quickly exercises these code paths, but edge coverage fails to distinguish the specific state combinations that trigger the bug. Without state-sensitive probes, the fuzzer misses the unusual setup (installing a destructive setter on a Worker property) and the vulnerable cleanup condition (a Worker entering finalizer execution with desynchronized internal references). By making these runtime states explicitly visible, our probes allow the fuzzer to detect and retain the novel combinations that lead to the crash.

Listing 11. Probe 2: Worker state at finalization (quickjs-libc.c).

```
1 static void js_worker_finalizer(JSRuntime *rt,
2     JSValue val) {
3   JSWorkerData *w = JS_GetOpaque(val, ...);
4   if (w) {
5     // gc phase x handler state
6     SEMANTIC_CONTEXT_ENUM(0x4016,
7       (uint32_t)(rt->gc_phase),
8       w->msg_handler
9         ? JS_VALUE_GET_TAG(
10            w->msg_handler->on_message_func)
11        : JS_TAG_NULL);
12    js_free_port(rt, w->msg_handler);
13  }
14 }
15 static void js_free_port(JSRuntime *rt,
16     JSWorkerMessageHandler *port) {
17   ...
18   list_del(&port->link);  // neighbours live in
19   js_free_rt(rt, port);   // the freed thread state
20 }
```

V8: Stale Addresses in the JIT Compiler. Unlike the QuickJS bug, which is related to the lifecycle of a single object, this V8 bug involves coordination across multiple subsystems. It was awarded a bug bounty by the V8 security team.

Listing 12 is a simplified version of the PoC. Lines 3–5 force a concatenated string into a globally shared pool by using it as a property key, while line 6 triggers JIT compilation. The vulnerability is triggered in lines 10–11: a Worker thread executes the JIT-compiled code while the main thread invokes a GC in the shared heap that relocates the string. Because this GC only updates the main thread's relocation counter, the Worker's compiler is unaware of the move and operates on a stale memory address, leading to a memory safety violation.

Listing 12. Simplified V8 PoC.

```
1 function entry() {
2   function f() {
3     let s = "codePointAt";
4     s += "toStringTag";  // stored in shared string pool
5     ("h")[s];            // used as property key
6     for (let i=0; i<1e6; i++) {} // trigger JIT
7   }
8   for (let i = 0; i < 100; i++) f();
9 }
10 new Worker(entry, {type:"function"});
11 gc();  // shared-heap GC relocates the string
```

This vulnerability highlights the complex interaction between the JIT compiler and garbage collector. Triggering it

requires two specific events, which our tool captures using targeted probes:

Listing 13. Probe 1: compiled constant type and location (js-graph.cc).

```
Node* JSGraph::HeapConstantNoHole(
    Handle<HeapObject> value) {
  SEMANTIC_CONTEXT_ENUM(0x23080,
    (uint32_t)(value->map()->instance_type()),
    (uint32_t)(MemoryChunk::FromHeapObject(
      *value)->owner_identity()));
  ...         // object type x memory region
}
```

Listing 13 is placed where the JIT compiler records a heap object as a constant in the compiled code. The probe hashes the object's type with the memory region it resides in. Most compiled constants are read-only objects (maps, builtins) that the GC never moves; when the compiler embeds a string that lives in the shared heap, the probe triggers a novel type-region combination that the fuzzer has not previously seen, thus retaining the input for further mutation.

Novel type-region combination alone is not sufficient to trigger the bug; the GC must also perform a compacting collection that can relocate the object. StateLens's second probe (Listing 14) hashes the GC collector type together with the collection reason. Most inputs trigger only minor, non-relocating collections; inputs triggering a full mark-compact collection trigger a novel state.

Listing 14. Probe 2: GC collector and reason (gc-tracer.cc).

```
RecordGCPhasesInfo(Heap* heap,
    GarbageCollector collector,
    GarbageCollectionReason reason) {
  SEMANTIC_CONTEXT_ENUM(0x31023,
    (uint32_t)(collector),
    (uint32_t)(reason)); // collector x reason
  ...
}
```

As such, by retaining inputs that cause both probes to hit novel states and mutating them further, StateLens can find the specific combination of a shared-space constant and a compacting GC that leads to the vulnerability. An edge-coverage fuzzer cannot distinguish *where* the compiler embeds objects or *which* strategy GC runs, thus cannot guide the input search towards the error-triggering combination.

6.3 Comparison with Baselines

Baselines. We compare StateLens against four state-of-the-art JavaScript engine fuzzers. Since StateLens is built on top of Fuzzilli [40], an IR-based generation fuzzer, the Fuzzilli comparison serves as a controlled ablation: the two tools share the same mutation engine, corpus generation strategy, and runtime infrastructure, differing only in StateLens's state-sensitive feedback. The remaining baselines are DIE [70], an aspect-preserving mutation fuzzer; OptFuzz [84], a JIT optimization-path-guided fuzzer; and HLPFuzz [96], an LLM-based constraint-solving fuzzer. OptFuzz targets JIT-specific behavior and is therefore evaluated only on the three JIT-enabled engines (V8, JSC, SpiderMonkey). We use the latest versions of each fuzzer at the time of evaluation with their default or recommended configurations. For fuzzers that require seed inputs, following prior work [70], we use the test suites shipped from each engine's repository as the initial corpus for fuzzing that engine. Note that StateLens does not require seed inputs.

Setup. To account for randomness, we evaluate each fuzzer-engine combination 5 times independently. Each of these runs executes the entire pipeline from scratch, including re-instrumentation of the target engine. Following common evaluation practices [84], each run uses 10 dedicated CPU cores and lasts for 72 hours.

Bug Comparison. We use bug-finding as the primary evaluation criterion, as it is an objective end-to-end measure independent of any coverage metric. We deduplicate crashes by manual root-cause analysis and report unique bugs per engine. Over the 72-hour evaluation window, StateLens discovers 39 unique bugs in total, while the second-best fuzzer, Fuzzilli, only finds 23 (a 70%↑). Notably, 14 bugs found by StateLens are not triggered by any baseline.

Coverage Comparison. We replay every fuzzer's final corpus on two common builds of each engine. The standard edge-coverage build contains no StateLens probes. The second build contains the same fixed probe set for every corpus and records unique entries in the shared-memory bitmap (Section 4.4). We normalize state coverage per engine to Fuzzilli, whose coverage is 1.0×.

As shown in Table 2, all fuzzers achieve comparable edge coverage. StateLens ranks first on three of the six engines, and its absolute gap from the best baseline remains below one percentage point on every engine. State coverage separates the fuzzers more clearly: DIE, OptFuzz, and HLPFuzz reach 1.01×, 1.23×, and 1.18× Fuzzilli's state coverage on average, whereas StateLens reaches 1.72×. The baselines therefore exercise fewer states despite reaching similar code. The 14 bugs found exclusively by StateLens require this state-coverage feedback. Even when edge coverage saturates, as in QuickJS, StateLens continues to trigger new semantic-bitmap entries corresponding to necessary vulnerability conditions such as those in Section 6.2.

6.4 Component Effectiveness

To understand the contribution of individual design choices, we conduct an ablation study by systematically varying three dimensions of StateLens: the language model used in the

Table 2. Coverage over 72h (median of 5 end-to-end runs). Edge: percentage of edges covered. State: semantic-bitmap entries relative to Fuzzilli (1.00×). Best result per engine is **bolded**. "N/A" indicates the fuzzer does not support that engine.

	V8		JSC		SM		QJS		Hermes		Escargot	
Fuzzer	**Edge**	**State**	**Edge**	**State**	**Edge**	**State**	**Edge**	**State**	**Edge**	**State**	**Edge**	**State**
Fuzzilli	15.30%	1.00	24.45%	1.00	**26.54%**	1.00	51.60%	1.00	25.37%	1.00	42.03%	1.00
DIE	13.12%	1.15	21.75%	1.04	20.39%	0.74	50.91%	1.03	24.52%	0.95	39.85%	1.17
OptFuzz	14.70%	1.48	**25.84%**	1.18	25.58%	1.04	N/A	N/A	N/A	N/A	N/A	N/A
HLPFuzz	14.24%	1.21	24.56%	1.13	23.51%	1.02	51.92%	1.10	**26.50%**	1.36	42.22%	1.27
StateLens	**15.85%**	**2.04**	24.92%	**2.19**	26.13%	**1.33**	**52.14%**	**1.30**	26.17%	**1.75**	**42.88%**	**1.68**

Table 3. Ablation study over 72 hours (median of five independent runs). Edge: percentage of edges covered. Bugs: unique bugs found.

	V8		QuickJS	
Variant	**Edge**	**Bugs**	**Edge**	**Bugs**
StateLens (full)	15.85%	14	52.14%	5
Language model				
w/ GPT-5 Mini	15.64%	11	52.01%	2
w/ GLM 4.7	15.70%	12	51.95%	4
Probe density				
50% probes	15.83%	10	52.34%	3
30% probes	15.78%	10	51.93%	2
10% probes	15.74%	9	51.82%	2
Knowledge base				
w/o KB	15.79%	10	52.14%	3

analysis, the number of inserted probes, and existence of the knowledge base. We evaluate on V8 and QuickJS, as these are two representative engines with distinct complexity and common use cases (server vs. embedded). All variants are executed under the same setup as in Section 6.3. Table 3 reports the results.

Language Model. To assess sensitivity to the underlying model, we replace GPT-5.1 with two alternatives: GPT-5 Mini [11] and GLM 4.7 [10] and re-run the whole pipeline to generate probes and fuzz with them. GPT-5.1 generates 1,194 probes on V8 and 289 on QuickJS, compared with 872/233 for GPT-5 Mini and 1,232/346 for GLM 4.7. GPT-5 Mini and GLM 4.7 uncover 13 (11 on V8, 2 on QuickJS) and 16 (12 on V8, 4 on QuickJS) bugs, respectively, compared with GPT-5.1's 19 (14 on V8, 5 on QuickJS).

Manual inspection of the generated probes reveals a qualitative difference. GPT-5.1 generates probes that target specific engine-internal semantics, isolating state transitions crucial for bug exposure, such as the exact type assumptions of JIT guards or object statuses at GC safepoints. Conversely, weaker models produce probes that capture only coarse-grained states, like function entry points or high-level branch conditions, and do not correlate with internal state transitions. Although all models generate compilable instrumentation, the disparity in bug discovery can stem from how the probes monitor relevant state changes.

Probe Density. To measure how the number of probes affects bug finding, we randomly retain 50%, 30%, and 10% of the original probes and re-evaluate fuzzing performance. As the probe count decreases, the number of bugs found also drops: At 50% retention, StateLens finds 10 bugs on V8 and 3 on QuickJS, a reduction of 29% and 40% respectively compared to the full probe set. At 30%, the counts drop further to 10 and 2. At 10%, only 9 and 2 bugs remain, a 42% overall reduction. Edge coverage, by contrast, remains stable across all variants, with less than 5% variation on V8.

Knowledge Base. To evaluate the contribution of the knowledge base, we disable the agent's access to the knowledge base and only provide the source code as context for probe generation. Disabling the knowledge base reduces probe counts by 22% on V8 and 34.5% on QuickJS, causing found bugs to decrease from 14 to 10 on V8 and from 5 to 3 on QuickJS. Manual inspection reveals this decline is qualitative as well: KB-free probes target shallow states like temporary variables and explicit branch conditions, while the full pipeline identifies state transitions and cross-component interactions generalized from historical bug patterns. The knowledge base thus steers the agent toward deeper state dimensions that are more likely to expose bugs.

7 Discussion

Testing vs. Verification. StateLens adopts the testing approach to enhance the security and reliability of JS engines. Another potential approach is to formally verify the correctness of JS engines. StateLens, like any other testing approach, cannot guarantee the absence of bugs. While formal verification offers the promise of mathematically proving software correctness, fully verifying all aspects of JS engines is still an open research problem due to their complexity (millions of lines of code with intricate semantics and optimizations). Testing, on the other hand, is more scalable and offers error-triggering inputs that facilitate debugging and patching; StateLens also achieves zero false positives in all

of the bugs it found. In line with other works for complex system reliability [27, 35, 36, 41, 54, 77, 79, 80], StateLens adopts testing as its main technique.

Extension to other JS Engines. A potential question is whether StateLens can generalize to other JS engines beyond the ones evaluated in this work. StateLens's design is not tied to a particular JS engine, and the LLM-guided instrumentation pipeline operates over C++ source and generic ASTs. StateLens is evaluated on a broad range of JS engines, including the default JS engines from major browsers (V8, SpiderMonkey, JavaScriptCore) and those used in embedded systems and IoT devices (QuickJS, Hermes, Escargot). We believe that extending to other JS engines requires minimal engineering effort.

LLM Agents for Vulnerability Discovery. LLM agents have begun to discover vulnerabilities on their own. Big Sleep [2] and Mythos [1] have disclosed bugs in widely deployed software such as the Linux kernel, FFmpeg, and OpenSSL, and the approach now spans OS kernels [55, 93], web applications [47, 76], smart contracts [58, 81], firmware and embedded systems [46], binaries [26, 57], reverse engineering [59, 87, 88], and penetration testing [32, 75]. Unlike StateLens, which invokes the model once offline to synthesize instrumentation, these systems keep the LLM in the loop as the reasoner that proposes and validates candidate vulnerabilities. To see whether this role difference matters in practice, we asked Codex to search QuickJS for vulnerabilities and produce runnable proofs of concept. The search yielded one genuine bug, which StateLens had also found, alongside several candidates that manual triage ruled out. This suggests the two approaches are complementary, and we leave their combination to future work.

8 Related Work

JavaScript Engine Fuzzing. LangFuzz [44], Jsfunfuzz [66], CodeAlchemist [42], Montage [52], Superion [85], DIE [70], and SoFi [43] mutate ASTs of existing test cases; TokenFuzz [73] and CovRL-Fuzz [33] mutate at the token level; TemuJS [89] performs template-based mutation. Fuzzilli [40] defines a custom IR (FuzzIL) that captures JS semantics and performs mutations and synthesis on top of it. OptFuzz [84] improves fuzzing efficiency through path coverage. A separate line of work targets logic bugs through better testing oracles like conformance testing [68, 98], differential testing [24, 83, 86], and JIT compiler validation [50]. StateLens provides state-sensitive feedback that captures a dimension that standard coverage metrics may miss, steering the fuzzer toward more diverse runtime behaviors; it is thus orthogonal to these works.

Language Virtual Machine Testing. JVM fuzzers have targeted bytecode validity via coverage-guided mutation [30] and rule-based generation [29], leveraged historical bug patterns to guide mutant synthesis [106, 107], and focused on JIT-specific bugs through optimization-activating mutators [90], joint program–option mutation [48], and cross-tier differential testing [53]. JAttack [101] and LeJit [100] use template-based generation to trigger JVM bugs, but their templates encode fixed syntactic patterns. In contrast, StateLens targets the engine's internal state directly by instrumenting state expressions identified from developer artifacts, providing semantic feedback orthogonal to code coverage. Beyond JVMs, other LVMs like WebAssembly runtimes [45, 60, 104], BPF verifiers [61, 72, 78], and Ethereum VMs [28, 34, 62, 95], have also been fuzzed. It would be promising to extend StateLens to complement the testing of other LVMs by providing state-sensitive feedback to explore more diverse program behavior.

LLM-Assisted Fuzzing. LLMs have shown strong capabilities in code understanding and program analysis [12, 71, 93]. A growing line of work applies LLMs to support fuzzing [69, 82, 94, 96, 103, 105], with most approaches using LLMs to directly synthesize test inputs for the target program [91, 92, 96] or to construct struct-aware harnesses or grammar that constrain the input search space [25, 65, 102]. StateLens takes a complementary approach: rather than using LLMs to generate inputs or harnesses, it leverages LLMs to synthesize instrumentation probes as guidance for the fuzzer.

9 Conclusion

We present StateLens, a state-aware fuzzing framework that finds JavaScript engine bugs beyond the reach of edge coverage. By using an LLM agent to mine developer artifacts and synthesize lightweight instrumentation, StateLens turns semantically meaningful internal states into fuzzing feedback without manual, per-engine annotation. Across six production JavaScript engines, StateLens uncovered 68 bugs, at least 35 of them with security implications. These results demonstrate the effectiveness and real-world impact of StateLens and offer a promising direction for testing and securing other large, stateful software systems.

Acknowledgment

We would like to thank the anonymous reviewers and our shepherd, Andi Quinn, for their valuable feedback. The HKUST authors were supported in part by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China HKUST C6004-25G and 16214723.

References

[1] Assessing Claude Mythos Preview's cybersecurity capabilities. https://www.anthropic.com/research/mythos-preview.

[2] From Naptime to Big Sleep: Using Large Language Models To Catch Vulnerabilities In Real-World Code. https://projectzero.google/2024/10/from-naptime-to-big-sleep.html.

[3] Security: Type confusion in v8 value serializer. https://issues.chromium.org/issues/40062884.

[4] The chromium super (inline cache) type confusion. https://github.blog/security/vulnerability-research/the-chromium-super-inline-cache-type-confusion/, 2022.

[5] Type confusion leading to rce in the chrome renderer sandbox. https://securitylab.github.com/advisories/GHSL-2024-095_Chromium/, 2024.

[6] Apache couchdb documentation: Query server. https://docs.couchdb.org/en/stable/query-server/index.html, 2025.

[7] Aws lambda runtimes. https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtimes.html, 2025.

[8] Bun documentation. https://bun.sh/docs#what-is-bun, 2025.

[9] Deno: The next-generation javascript runtime. https://deno.com/, 2025.

[10] Glm-4.7 overview. https://docs.z.ai/guides/llm/glm-4.7, 2025.

[11] Gpt-5 mini model. https://developers.openai.com/api/docs/models/gpt-5-mini, 2025.

[12] Gpt-5.1 model. https://developers.openai.com/api/docs/models/gpt-5.1, 2025.

[13] HKCERT CTF 2025 Piano. https://github.com/hkcert-ctf/CTF-Challenges/tree/main/CTF-2025/piano, 2025.

[14] How workers works. https://developers.cloudflare.com/workers/reference/how-workers-works/, 2025.

[15] Mongodb documentation: Server-side javascript. https://www.mongodb.com/docs/manual/core/server-side-javascript/, 2025.

[16] Node.js — run javascript everywhere. https://nodejs.org/, 2025.

[17] React native documentation: Javascript runtime. https://reactnative.dev/docs/hermes/, 2025.

[18] Safe in the sandbox: security hardening for cloudflare workers. https://blog.cloudflare.com/safe-in-the-sandbox-security-hardening-for-cloudflare-workers/, 2025.

[19] [turbofan] fix transitionelementskindorcheckmap. https://chromium.googlesource.com/v8/v8/+/8b490a9690b859346a68a3d2a7008b4e1852c3ea, 2025.

[20] [wasm] replace dead_code_ set with is_dying_ bit. https://chromium.googlesource.com/v8/v8/+/33ca4f51e5dbba9817eba16fd3249e66a880cf33, 2025.

[21] Abdullah Alhamdan and Cristian-Alexandru Staicu. SandDriller: A Fully-Automated approach for testing Language-Based JavaScript sandboxes. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3457–3474, 2023.

[22] Mir Masood Ali, Mohammad Ghasemisharif, Chris Kanich, and Jason Polakis. Rise of inspectron: Automated black-box auditing of cross-platform electron apps. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 775–792, 2024.

[23] Cornelius Aschermann, Sergej Schumilo, Ali Abbasi, and Thorsten Holz. Ijon: Exploring deep state spaces via fuzzing. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1597–1612. IEEE, 2020.

[24] Lukas Bernhard, Tobias Scharnowski, Moritz Schloegel, Tim Blazytko, and Thorsten Holz. Jit-picking: Differential fuzzing of javascript engines. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 351–364, 2022.

[25] Chuyang Chen, Brendan Dolan-Gavitt, and Zhiqiang Lin. *ELFUZZ: efficient input generation via LLM-driven synthesis over fuzzer space.* SEC '25. 2025.

[26] Xiang Chen, Anshunkang Zhou, Chengfeng Ye, and Charles Zhang. Clearagent: Agentic binary analysis for effective vulnerability detection. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages*, 2025.

[27] Yinfang Chen, Xudong Sun, Suman Nath, Ze Yang, and Tianyin Xu. {Push-Button} reliability testing for {Cloud-Backed} applications with rainmaker. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1701–1716, 2023.

[28] Yuanliang Chen, Fuchen Ma, Yuanhang Zhou, Yu Jiang, Ting Chen, and Jiaguang Sun. Tyr: Finding Consensus Failure Bugs in Blockchain System with Behaviour Divergent Model. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2517–2532, 2023.

[29] Yuting Chen, Ting Su, and Zhendong Su. Deep differential testing of jvm implementations. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 1257–1268. IEEE, 2019.

[30] Yuting Chen, Ting Su, Chengnian Sun, Zhendong Su, and Jianjun Zhao. Coverage-directed differential testing of jvm implementations. In *proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 85–99, 2016.

[31] Chrome Vulnerability Reward Program Rules. https://bughunters.google.com/about/rules/chrome-friends/chrome-vulnerability-reward-program-rules.

[32] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, 2024.

[33] Jueon Eom, Seyeon Jeong, and Taekyoung Kwon. Fuzzing JavaScript Interpreters with Coverage-Guided Reinforcement Learning for LLM-Based Mutation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, 2024.

[34] Ying Fu, Meng Ren, Fuchen Ma, Heyuan Shi, Xin Yang, Yu Jiang, Huizhong Li, and Xiang Shi. Evmfuzzer: detect evm vulnerabilities via fuzz testing. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019.

[35] Sishuai Gong, Dinglan Peng, Deniz Altınbüken, Pedro Fonseca, and Petros Maniatis. Snowcat: Efficient kernel concurrency testing using a learned coverage predictor. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 35–51, 2023.

[36] Sishuai Gong, Wang Rui, Deniz Altinbüken, Pedro Fonseca, and Petros Maniatis. Snowplow: Effective kernel fuzzing with a learned white-box test mutator. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1124–1138, 2025.

[37] Google. 0day "in the wild". https://docs.google.com/spreadsheets/d/1lkNJ0uQwbeC1ZTRrxdtuPLCIl7mlUreoKfSIgajnSyY/, 2025.

[38] Google Security Research. Public v8ctf submissions. https://docs.google.com/spreadsheets/d/e/2PACX-1vTWvO0tFNl8fJbOmTV1nwGJi4fAy5pDg-6DsHARRubj8I6c7_11RQ36Jv735zj9EQggz6AWjAOaebJh/pubhtml.

[39] Google Security Research. v8CTF Rules. https://github.com/google/security-research/blob/master/v8ctf/rules.md.

[40] Samuel Groß, Simon Koch, Lukas Bernhard, Thorsten Holz, and Martin Johns. FUZZILLI: Fuzzing for JavaScript JIT Compiler Vulnerabilities. In *Proceedings 2023 Network and Distributed System Security Symposium*, San Diego, CA, USA, 2023. Internet Society.

[41] Jiawei Tyler Gu, Xudong Sun, Wentao Zhang, Yuxuan Jiang, Chen Wang, Mandana Vaziri, Owolabi Legunsen, and Tianyin Xu. Acto: Automatic end-to-end testing for operation correctness of cloud system management. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 96–112, 2023.

[42] HyungSeok Han, DongHyeon Oh, and Sang Kil Cha. Codealchemist: Semantics-aware code generation to find vulnerabilities in javascript engines. In *NDSS*, 2019.

[43] Xiaoyu He, Xiaofei Xie, Yuekang Li, Jianwen Sun, Feng Li, Wei Zou, Yang Liu, Lei Yu, Jianhua Zhou, Wenchang Shi, et al. Sofi: Reflection-augmented fuzzing for javascript engines. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2229–2242, 2021.

[44] Christian Holler, Kim Herzig, and Andreas Zeller. Fuzzing with code fragments. In *21st USENIX Security Symposium (USENIX Security 12)*,

pages 445–458, 2012.

[45] Yage Hu, Wen Zhang, Botang Xiao, Qingchen Kong, Boyang Yi, Suxin Ji, Songlan Wang, and Wenwen Wang. Wasit: Deep and continuous differential testing of webassembly system interface implementations. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, 2025.

[46] Jiangan Ji, Chao Zhang, Shuitao Gan, Lin Jian, Hangtian Liu, Tieming Liu, Lei Zheng, and Zhipeng Jia. Firmagent: Leveraging fuzzing to assist llm agents with iot firmware vulnerability discovery. In *NDSS*, 2026.

[47] Yuchen Ji, Ting Dai, Zhichao Zhou, Yutian Tang, and Jingzhu He. Artemis: Toward accurate detection of server-side request forgeries through llm-assisted inter-procedural path-sensitive taint analysis. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA1):1349–1377, 2025.

[48] Haoxiang Jia, Ming Wen, Zifan Xie, Xiaochen Guo, Rongxin Wu, Maolin Sun, Kang Chen, and Hai Jin. Detecting jvm jit compiler bugs via exploring two-dimensional input spaces. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023.

[49] Yuancheng Jiang, Chuqi Zhang, Bonan Ruan, Jiahao Liu, Manuel Rigger, Roland HC Yap, and Zhenkai Liang. Fuzzing the PHP interpreter via dataflow fusion. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6143–6158, 2025.

[50] Seungwan Kwon, Jaeseong Kwon, Wooseok Kang, Juneyoung Lee, and Kihong Heo. Translation validation for jit compiler in the v8 javascript engine. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12, 2024.

[51] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*

[52] Suyoung Lee, HyungSeok Han, Sang Kil Cha, and Sooel Son. Montage: A neural network language model-guided javascript engine fuzzer. In *29th USENIX Security Symposium (USENIX Security 20)*, 2020.

[53] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. Validating jit compilers via compilation space exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 66–79, 2023.

[54] Guangpu Li, Shan Lu, Madanlal Musuvathi, Suman Nath, and Rohan Padhye. Efficient scalable thread-safety-violation detection: finding thousands of concurrency bugs during testing. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

[55] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. Enhancing static analysis for practical bug detection: An llm-integrated approach. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1), 2024.

[56] Penghui Li, Wei Meng, and Chao Zhang. SDFuzz: Target states driven directed fuzzing. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, aug 2024.

[57] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhenyang Xu, Zhi Li, Peng Di, Yu Jiang, et al. Llm-powered static binary taint analysis. *ACM Transactions on Software Engineering and Methodology*, 34(3):1–36, 2025.

[58] Ye Liu, Yue Xue, Daoyuan Wu, Yuqiang Sun, Yi Li, Miaolei Shi, and Yang Liu. Propertygpt: Llm-driven formal verification of smart contracts through retrieval-augmented property generation. *arXiv preprint arXiv:2405.02580*, 2024.

[59] Zhibo Liu, Huaijin Wang, Wai Kin Wong, Daoyuan Wu, and Shuai Wang. No more translation at runtime: Llm-empowered static binary translation. In *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026*. ACM, 2026.

[60] Zhibo Liu, Dongwei Xiao, Zongjie Li, Shuai Wang, and Wei Meng. Exploring missed optimizations in webassembly optimizers. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 436–448, 2023.

[61] Tao Lyu, Kumar Kartikeya Dwivedi, Thomas Bourgeat, Mathias Payer, Meng Xu, and Sanidhya Kashyap. ebpf misbehavior detection: Fuzzing with a specification-based oracle. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, 2025.

[62] Fuchen Ma, Yuanliang Chen, Meng Ren, Yuanhang Zhou, Yu Jiang, Ting Chen, Huizhong Li, and Jiaguang Sun. LOKI: State-Aware Fuzzing Framework for the Implementation of Blockchain Consensus Protocols. In *Proceedings 2023 Network and Distributed System Security Symposium*, San Diego, CA, USA, 2023. Internet Society.

[63] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: iterative refinement with self-feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023.

[64] Philipp Mao, Marcel Busch, and Mathias Payer. NASS: Fuzzing all native android system services with interface awareness and coverage. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 4225–4243, 2025.

[65] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.

[66] Mozilla. Github - mozillasecurity/funfuzz: A collection of fuzzers in a harness for testing the spidermonkey javascript engine. https://github.com/MozillaSecurity/funfuzz/tree/master, 2023.

[67] OSS Fuzz. https://issues.oss-fuzz.com/issues.

[68] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. Jest: N+ 1-version differential testing of both javascript engines and specification. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021.

[69] Junyoung Park and Insu Yun. Agentic fuzzing: Opportunities and challenges. *arXiv preprint arXiv:2605.10074*, 2026.

[70] Soyeon Park, Wen Xu, Insu Yun, Daehee Jang, and Taesoo Kim. Fuzzing javascript engines with aspect-preserving mutation. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020.

[71] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[72] Chaoyuan Peng, Muhui Jiang, Lei Wu, and Yajin Zhou. Toss a fault to bpfchecker: Revealing implementation flaws for ebpf runtimes with differential fuzzing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, New York, NY, USA, 2024. Association for Computing Machinery.

[73] Christopher Salls, Chani Jindal, Jake Corina, Christopher Kruegel, and Giovanni Vigna. Token-Level fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, aug 2021.

[74] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. {AddressSanitizer}: A fast address sanity checker. In *2012 USENIX annual technical conference (USENIX ATC 12)*, 2012.

[75] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous llm-assisted system for red teaming multi-host networks. In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 4282–4300. IEEE, 2026.

[76] Aleksei Stafeev, Tim Recktenwald, Gianluca De Stefano, Soheil Khodayari, and Giancarlo Pellegrino. Yurascanner: Leveraging llms for task-driven web app scanning. In *NDSS*, 2025.

[77] Bogdan Alexandru Stoica, Shan Lu, Madanlal Musuvathi, and Suman Nath. Waffle: exposing memory ordering bugs efficiently with active delay injection. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 111–126, 2023.

[78] Hao Sun and Zhendong Su. Validating the ebpf verifier via state embedding. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, OSDI'24, USA, 2024. USENIX Association.

[79] Xudong Sun, Runxiang Cheng, Jianyan Chen, Elaine Ang, Owolabi Legunsen, and Tianyin Xu. Testing configuration changes in context to prevent production failures. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.

[80] Xudong Sun, Wenqing Luo, Jiawei Tyler Gu, Aishwarya Ganesan, Ramnatthan Alagappan, Michael Gasch, Lalith Suresh, and Tianyin Xu. Automatic reliability testing for cluster management controllers. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 143–159, 2022.

[81] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*, pages 1–13, 2024.

[82] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. Cottontail: Large language model-driven concolic execution for highly structured test input generation. In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 3509–3527. IEEE, 2026.

[83] Liam Wachter, Julian Gremminger, Christian Wressnegger, Mathias Payer, and Flavio Toffalini. Dumpling: Fine-grained differential javascript engine fuzzing. In *Proceedings 2025 Network and Distributed System Security Symposium*. Internet Society, 2025.

[84] Jiming Wang, Yan Kang, Chenggang Wu, Yuhao Hu, Yue Sun, Jikai Ren, Yuanming Lai, Mengyao Xie, Charles Zhang, Tao Li, et al. Optfuzz: optimization path guided fuzzing for javascript jit compilers. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 865–882. USENIX Association, 2024.

[85] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: Grammar-Aware Greybox Fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019.

[86] Junjie Wang, Zhiyi Zhang, Shuang Liu, Xiaoning Du, and Junjie Chen. Fuzzjit: Oracle-enhanced fuzzing for javascript engine jit compiler. In *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

[87] Wai Kin Wong, Daoyuan Wu, Zhibo Liu, Huaijin Wang, Zongjie Li, and Shuai Wang. Binrag: An rag-based decompilation framework fusing name prediction and calling context. *Proceedings of the ACM on Software Engineering*, 3(ISSTA), 2026.

[88] Wai Kin Wong, Daoyuan Wu, Huaijin Wang, Zongjie Li, Zhibo Liu, Shuai Wang, Qiyi Tang, Sen Nie, and Shi Wu. Decllm: Llm-augmented recompilable decompilation for enabling programmatic use of decompiled code. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1841–1864, 2025.

[89] Wai Kin Wong, Dongwei Xiao, Cheuk Tung Lai, Yiteng Peng, Daoyuan Wu, and Shuai Wang. Extraction and mutation at a high level: Template-based fuzzing for javascript engines. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):2898–2926, 2025.

[90] Mingyuan Wu, Minghai Lu, Heming Cui, Junjie Chen, Yuqun Zhang, and Lingming Zhang. Jitfuzz: Coverage-guided fuzzing for jvm just-in-time compilers. In *2023 IEEE/acm 45th international conference on software engineering (icse)*, pages 56–68. IEEE, 2023.

[91] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. ICSE '24, 2024.

[92] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. Whitefox: White-box compiler fuzzing empowered by large language models. (OOPSLA2), 2024.

[93] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. Knighter: Transforming static analysis with llm-synthesized checkers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025.

[94] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. Kernelgpt: Enhanced kernel fuzzing via large language models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025.

[95] Youngseok Yang, Taesoo Kim, and Byung-Gon Chun. Finding Consensus Bugs in Ethereum via Multi-transaction Differential Fuzzing. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 349–365, 2021.

[96] Yupeng Yang, Shenglong Yao, Jizhou Chen, and Wenke Lee. Hybrid language processor fuzzing via llm-based constraint solving. In *Proceedings of the 34th USENIX Conference on Security Symposium*, USA, 2025. USENIX Association.

[97] Zheng Yang, Simon P Chung, Jizhou Chen, Runze Zhang, Brendan Saltaformaggio, and Wenke Lee. Coindef: a comprehensive code injection defense for the electron framework. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3127–3144. IEEE, 2025.

[98] Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Xiaoyang Sun, Lizhong Bian, Haibo Wang, and Zheng Wang. Automated conformance testing for javascript engines via deep compiler fuzzing. In *Proceedings of the 42nd ACM SIGPLAN international conference on programming language design and implementation*, pages 435–450, 2021.

[99] Michal Zalewski. american fuzzy lop - a security-oriented fuzzer. https://github.com/google/afl, 2025.

[100] Zhiqiang Zang, Fu-Yao Yu, Aditya Thimmaiah, August Shi, and Milos Gligoric. Java jit testing with template extraction. *Proceedings of the ACM on Software Engineering*, 1(FSE):1129–1151, 2024.

[101] Zhiqiang Zang, Fu-Yao Yu, Nathan Wiatrek, Milos Gligoric, and August Shi. Jattack: Java jit testing using template programs. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 6–10. IEEE, 2023.

[102] Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. Low-cost and comprehensive non-textual input fuzzing with llm-synthesized input generators. In *Proceedings of the 34th USENIX Conference on Security Symposium*, SEC '25, 2025.

[103] Kunpeng Zhang, Dongwei Xiao, Daoyuan Wu, Shuai Wang, Jiali Zhao, Yuanyi Lin, Tongtong Xu, and Shaohua Wang. Llm-powered silent bug fuzzing in deep learning libraries via versatile and controlled bug transfer. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1):1599–1626, 2026.

[104] Lingming Zhang, Binbin Zhao, Jiacheng Xu, Peiyu Liu, Qinge Xie, Yuan Tian, Jianhai Chen, and Shouling Ji. Waltzz: webassembly runtime fuzzing with stack-invariant transformation. In *Proceedings of the 34th USENIX Conference on Security Symposium*, 2025.

[105] Zhiyu Zhang, Longxing Li, Ruigang Liang, and Kai Chen. Unlocking low frequency syscalls in kernel fuzzing with dependency-based rag. (ISSTA), 2025.

[106] Yingquan Zhao, Zan Wang, Junjie Chen, Ruifeng Fu, Yanzhou Lu, Tianchang Gao, and Haojie Ye. Program ingredients abstraction and instantiation for synthesis-based jvm testing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3943–3957, 2024.

[107] Yingquan Zhao, Zan Wang, Junjie Chen, Mengdi Liu, Mingyuan Wu, Yuqun Zhang, and Lingming Zhang. History-driven test program synthesis for jvm testing. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1133–1144, 2022.

[108] Jie Zhu, Chihao Shen, Ziyang Li, Jiahao Yu, Yizheng Chen, and Kexin Pei. Locus: Agentic predicate synthesis for directed fuzzing. *arXiv preprint arXiv:2508.21302*, 2025.